# Module 1:
# Introduction to Graphs

Dr. Nirnay Ghosh

# Formal Definition & Examples

A *graph* $G = (V, E)$ consists of $V$, a nonempty set of *vertices* (or *nodes*) and $E$, a set of *edges*. Each edge has either one or two vertices associated with it, called its *endpoints*. An edge is said to *connect* its endpoints.

- Examples of graph in real life:




**FIGURE 1** **A Computer Network.**

- Simple graph:
  - No edge connects a vertex to itself
  - Each edge connects two different vertices
  - No two edge connects same pair of vertices
  - Each edge is associated to an unordered pair of vertices

# Formal Definition & Examples (*Contd...*)




**FIGURE 2** **A Computer Network with Multiple Links between Data Centers.**

- Multigraphs
  - Multiple edges connecting the same vertices

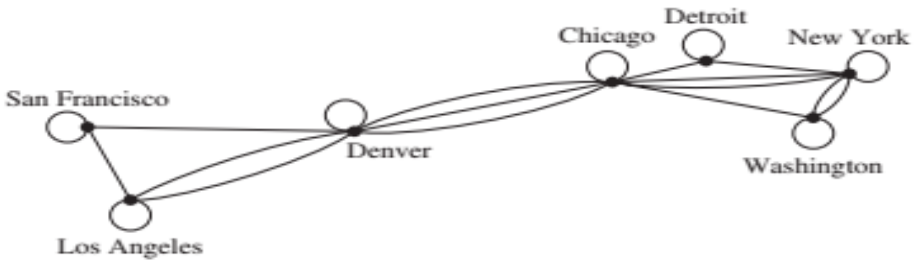



**FIGURE 3** **A Computer Network with Diagnostic Links.**

- Pseudographs
  - **Loops** - edges connect a vertex to itself
  - Multiple edges may connect the same pair of vertices or vertex

# Directed Graphs

A *directed graph* (or *digraph*) $(V, E)$ consists of a nonempty set of vertices $V$ and a set of *directed edges* (or *arcs*) $E$. Each directed edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair $(u, v)$ is said to *start* at $u$ and *end* at $v$.




FIGURE 4 A Communications Network with One-Way Communications Links.




FIGURE 5 A Computer Network with Multiple One-Way Links.

TABLE 1 Graph Terminology.

| *Type* | *Edges* | *Multiple Edges Allowed?* | *Loops Allowed?* |
|---|---|---|---|
| Simple graph | Undirected | No | No |
| Multigraph | Undirected | Yes | No |
| Pseudograph | Undirected | Yes | Yes |
| Simple directed graph | Directed | No | No |
| Directed multigraph | Directed | Yes | Yes |
| Mixed graph | Directed and undirected | Yes | Yes |

- Key questions
  - Are all edges of the graph undirected or directed (or both)?
  - Do multiple edges/directed edges connect the same pair of vertices?
  - Are loops present?

# Application of Graphs

- Social media
  - Acquaintanceship/Friendship graph
  - Influence graph

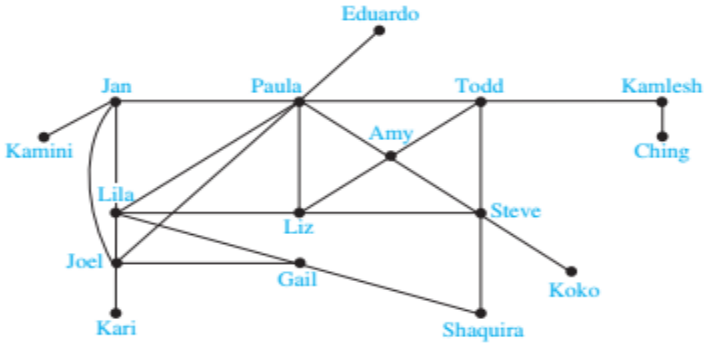



**FIGURE 6** An Acquaintanceship Graph.

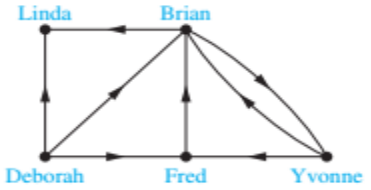



**FIGURE 7** An Influence Graph.

- Communication networks
  - Call graphs

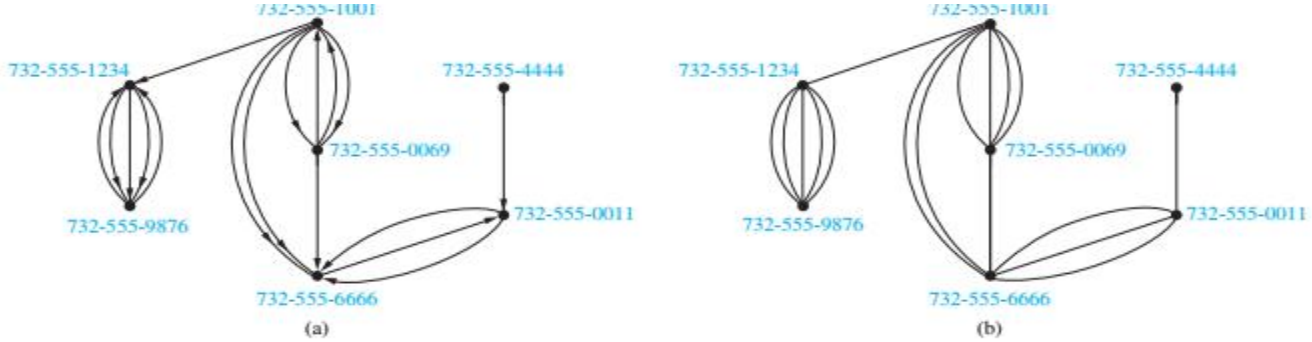



**FIGURE 8** A Call Graph.

# Application of Graphs (*Contd...*)

- Information networks
  - The Web Graph
  - Citation graphs
- Transportation networks
  - Airline routes
  - Road networks
- Software design applications
  - Module dependency graphs
  - Precedence graphs and concurrent processing

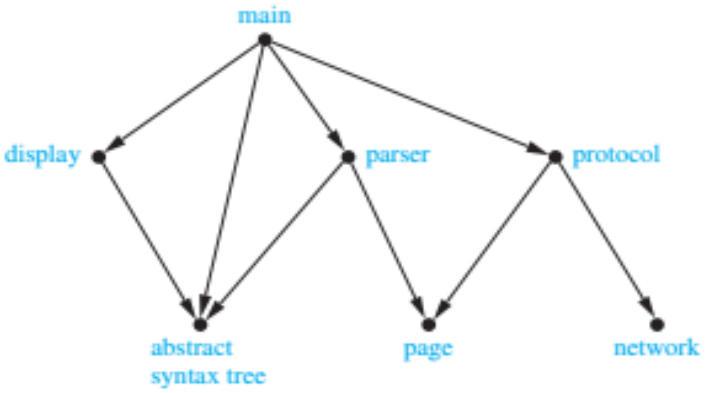



**FIGURE 9** **A Module Dependency Graph.**

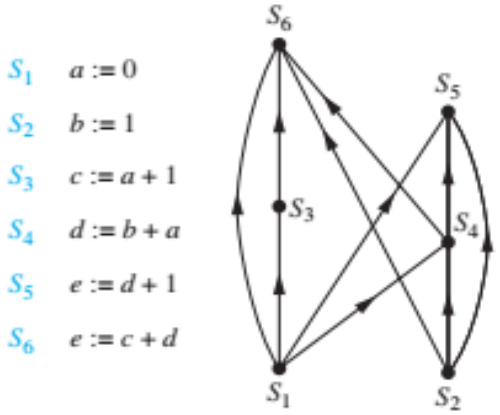



**FIGURE 10** **A Precedence Graph.**

# Application of Graphs (*Contd...*)

- Tournaments
  - Round-robin tournaments
  - Single-elimination tournaments

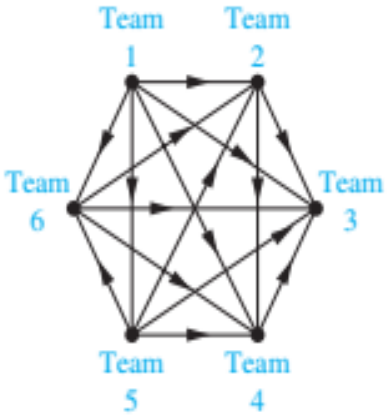



**FIGURE 13** **A Graph Model of a Round-Robin Tournament.**

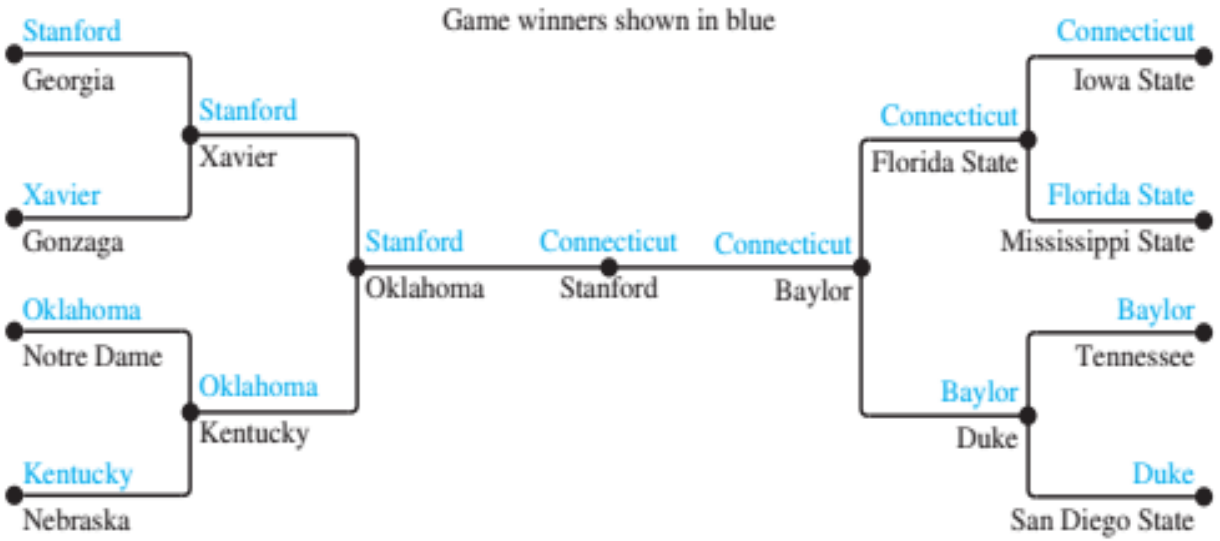



**FIGURE 14** **A Single-Elimination Tournament.**

# Graph Terminologies

- Terminologies to describe vertices and edges of undirected graph

Two vertices $u$ and $v$ in an undirected graph $G$ are called *adjacent* (or *neighbors*) in $G$ if $u$ and $v$ are endpoints of an edge $e$ of $G$. Such an edge $e$ is called *incident with* the vertices $u$ and $v$ and $e$ is said to *connect* $u$ and $v$.

The set of all neighbors of a vertex $v$ of $G = (V, E)$, denoted by $N(v)$, is called the *neighborhood* of $v$. If $A$ is a subset of $V$, we denote by $N(A)$ the set of all vertices in $G$ that are adjacent to at least one vertex in $A$. So, $N(A) = \bigcup_{v \in A} N(v)$.

The *degree of a vertex in an undirected graph* is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex. The degree of the vertex $v$ is denoted by $\deg(v)$.

- Isolated vertex: a vertex of degree zero (not adjacent to any of the other vertices)
- Pendant vertex: a vertex is pendant if and only if it has degree one (adjacent to exactly one vertex)

# Graph Terminologies (*Contd...*)

**Theorem 1:**

**THE HANDSHAKING THEOREM** Let $G = (V, E)$ be an undirected graph with $m$ edges. Then

$$2m = \sum_{v \in V} \deg(v).$$

(Note that this applies even if multiple edges and loops are present.)

**Theorem 2:**

An undirected graph has an even number of vertices of odd degree.

**Definition:**

When $(u, v)$ is an edge of the graph $G$ with directed edges, $u$ is said to be *adjacent to* $v$ and $v$ is said to be *adjacent from* $u$. The vertex $u$ is called the *initial vertex* of $(u, v)$, and $v$ is called the *terminal* or *end vertex* of $(u, v)$. The initial vertex and terminal vertex of a loop are the same.

# Graph Terminologies (*Contd...*)

- The edges in graphs with directed edges are ordered pairs
- The definition of the degree of a vertex can be refined to reflect the number of edges with this vertex as the initial vertex and as the terminal vertex
- Definition:

In a graph with directed edges the *in-degree of a vertex* $v$, denoted by $\deg^-(v)$, is the number of edges with $v$ as their terminal vertex. The *out-degree of* $v$, denoted by $\deg^+(v)$, is the number of edges with $v$ as their initial vertex. (Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of this vertex.)

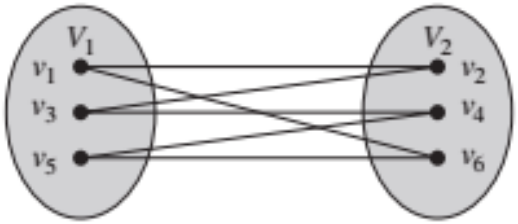
- Theorem:

Let $G = (V, E)$ be a graph with directed edges. Then

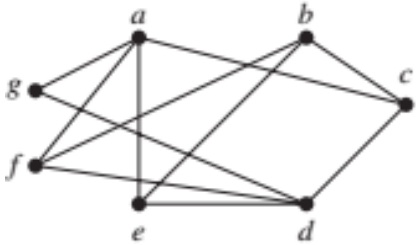
$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|.$$

# Bipartite Graph

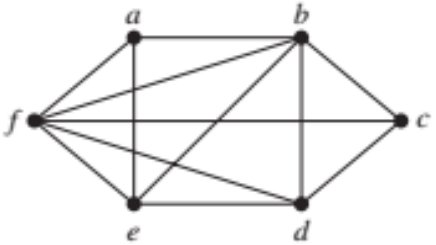
A simple graph $G$ is called *bipartite* if its vertex set $V$ can be partitioned into two disjoint sets $V_1$ and $V_2$ such that every edge in the graph connects a vertex in $V_1$ and a vertex in $V_2$ (so that no edge in $G$ connects either two vertices in $V_1$ or two vertices in $V_2$). When this condition holds, we call the pair $(V_1, V_2)$ a *bipartition* of the vertex set $V$ of $G$.




- Are the graphs $G$ and $H$ bipartite?




$G$




$H$

- Theorem:

A simple graph is bipartite if and only if it is possible to assign one of two different colors to each vertex of the graph so that no two adjacent vertices are assigned the same color.

# Bipartite Graph (*Contd...*)

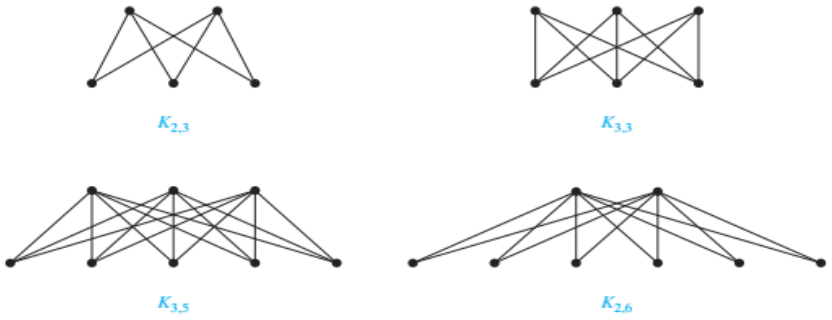
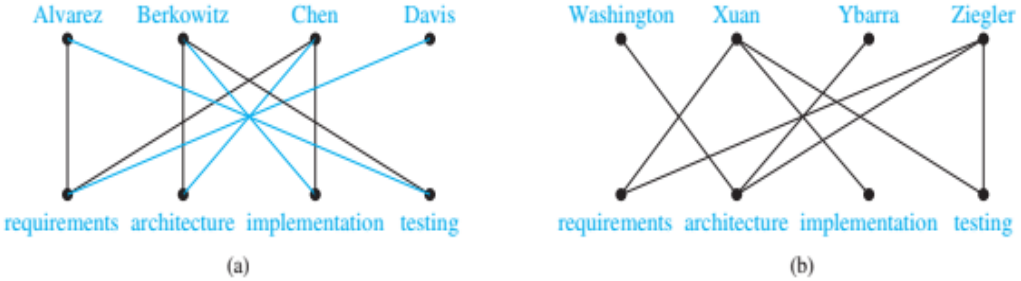



**FIGURE 9** **Some Complete Bipartite Graphs.**

- Application of bipartite graphs
  - Job assignment

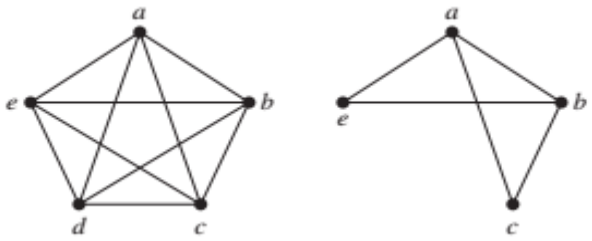



- Matching
  - Maximum matching
  - Complete matching from $V_1$ to $V_2$

**Marriages on an Island** Suppose that there are $m$ men and $n$ women on an island. Each person has a list of members of the opposite gender acceptable as a spouse. We construct a bipartite graph $G = (V_1, V_2)$ where $V_1$ is the set of men and $V_2$ is the set of women so that there is an edge between a man and a woman if they find each other acceptable as a spouse. A matching in this graph consists of a set of edges, where each pair of endpoints of an edge is a husband-wife pair. A maximum matching is a largest possible set of married couples, and a complete matching of $V_1$ is a set of married couples where every man is married, but possibly not all women.

# Subgraph

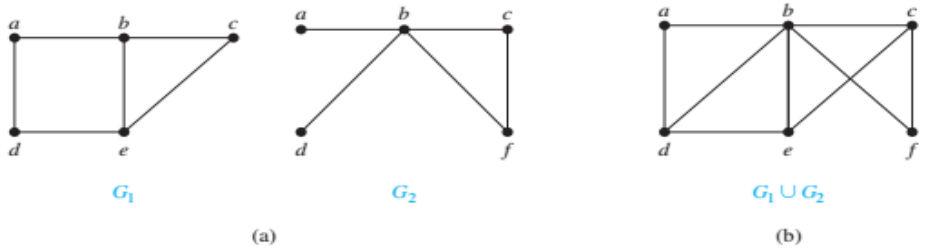
A *subgraph of a graph* $G = (V, E)$ is a graph $H = (W, F)$, where $W \subseteq V$ and $F \subseteq E$. A subgraph $H$ of $G$ is a *proper subgraph* of $G$ if $H \neq G$.




- The graph is a subgraph of $K_5$.
- If we add the edge connecting $c$ and $e$ to $G$, we obtain the subgraph induced by $W = \{a, b, c, e\}$.

Let $G = (V, E)$ be a simple graph. The **subgraph induced** by a subset $W$ of the vertex set $V$ is the graph $(W, F)$, where the edge set $F$ contains an edge in $E$ if and only if both endpoints of this edge are in $W$.

The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of $G_1$ and $G_2$ is denoted by $G_1 \cup G_2$.



# Graph Representation

- **Adjacency lists**: Specify the vertices that are adjacent to each vertex of the graph.

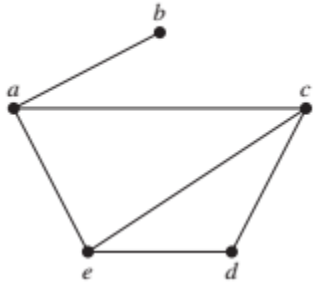



**TABLE 1** An Adjacency List for a Simple Graph.

| Vertex | Adjacent Vertices |
|---|---|
| a | b, c, e |
| b | a |
| c | a, d, e |
| d | c, e |
| e | a, c, d |

**Adjacency list representation of a simple graph**

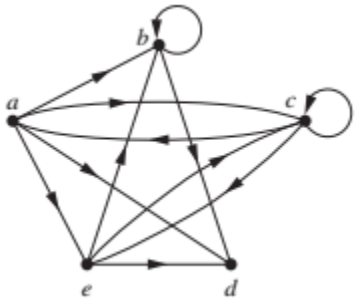


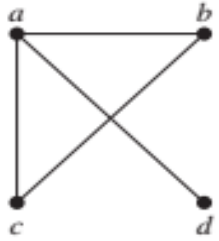

**TABLE 2** An Adjacency List for a Directed Graph.

| Initial Vertex | Terminal Vertices |
|---|---|
| a | b, c, d, e |
| b | b, d |
| c | a, c, e |
| d | |
| e | b, c, d |

**Adjacency list representation of a directed graph**

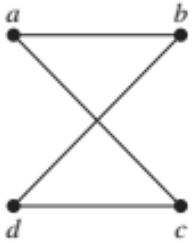
- **Adjacency matrices**: The **adjacency matrix A** (or $\mathbf{A}_G$) of $G$ is an $n$ x $n$ zero-one matrix with 1 as its $(i, j)^{th}$ entry when $v_i$ and $v_j$ are adjacent, and 0 as its $(i,j)^{th}$ entry when they are not adjacent. In other words, if its adjacency matrix is $\mathbf{A} = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

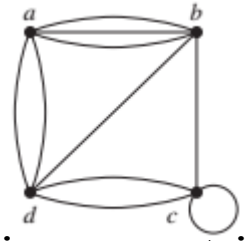



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

**Adjacency matrix representation of a simple graphs**




$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$




$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

**Adjacency matrix representation of a pseudo graph**

# Graph Representation (*Contd...*)

- **Adjacency matrix for directed graph**:
  - The matrix for a directed graph $G = (V, E)$ has a 1 in its $(i, j)$th position if there is an edge from $v_i$ to $v_j$, where $v_1, v_2, \ldots, v_n$ is an arbitrary listing of the vertices of the directed graph.
  - The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from $v_j$ to $v_i$ when there is an edge from $v_i$ to $v_j$
  - If $\mathbf{A} = [aij]$ is the adjacency matrix for the directed graph with respect to this listing of the vertices, then

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

  - Trade-off between Adjacency lists and Adjacency matrices:
    - Sparse graph: adjacency list
    - Dense graph: adjacency matrix

- **Incidence matrix:** Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \ldots, v_n$ are the vertices and $e_1, e_2, \ldots, e_m$ are the edges of $G$. Then the incidence matrix with respect to this ordering of $V$ and $E$ is the $n \times m$ matrix $M = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

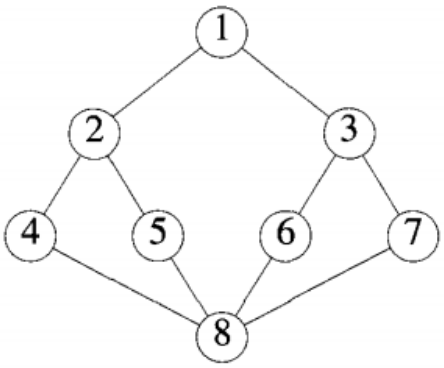
# Isomorphism of Graphs

The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there exists a one-to-one and onto function $f$ from $V_1$ to $V_2$ with the property that $a$ and $b$ are adjacent in $G_1$ if and only if $f(a)$ and $f(b)$ are adjacent in $G_2$, for all $a$ and $b$ in $V_1$. Such a function $f$ is called an *isomorphism*.* Two simple graphs that are not isomorphic are called *nonisomorphic*.

- *-The word isomorphism comes from the Greek roots *isos* for "equal" and *morphe* for "form".
- If two simple graphs are isomorphic, there is a one-to-one correspondence between vertices of the two graphs that **preserves the adjacency relationship.**
- Determining whether two simple graphs are isomorphic?
  - Hard problem, as there can be *n!* possible one-to-one correspondence between two simple graphs with *n* vertices
  - Easier to show if the graphs are *not* isomorphic by checking the following attributes of the *graph invariant* property:
    - Same number of vertices
    - Same number of edges
    - Degrees of the vertices must be the same

# Breadth First Search and Traversal (BFS/BFT)

```
1   Algorithm BFT(G, n)
2   // Breadth first traversal of G
3   {
4       for i := 1 to n do  // Mark all vertices unvisited.
5           visited[i] := 0;
6       for i := 1 to n do
7           if (visited[i] = 0) then BFS(i);
8   }
```




(a) Undirected graph $G$

Add $u$ to $q$

```
1   Algorithm BFS(v)
2   // A breadth first search of G is carried out beginning
3   // at vertex v. For any node i, visited[i] = 1 if i has
4   // already been visited. The graph G and array visited[ ]
5   // are global; visited[ ] is initialized to zero.
6   {
7       u := v; // q is a queue of unexplored vertices.
8       visited[v] := 1;
9       repeat
10      {
11          for all vertices w adjacent from u do
12          {
13              if (visited[w] = 0) then
14              {
15                  Add w to q; // w is unexplored.
16                  visited[w] := 1;
17              }
18          }
19          if q is empty then return; // No unexplored vertex.
20          Delete u from q; // Get first unexplored vertex.
21      } until(false);
22  }
```

# Depth First Search and Traversal (DFS/DFT)

```
1   Algorithm DFS(v)
2   // Given an undirected (directed) graph G = (V, E) with
3   // n vertices and an array visited[ ] initially set
4   // to zero, this algorithm visits all vertices
5   // reachable from v. G and visited[ ] are global.
6   {
7       visited[v] := 1;
8       for each vertex w adjacent from v do
9       {
10          if (visited[w] = 0) then DFS(w);
11      }
12  }
```

- Space Complexity:

| Implementation | BFS | DFS |
|---|---|---|
| Adjacency list | O(n) | O(n) |
| Adjacency Matrix | O(n) | O(n) |

- Time Complexity:

| Implementation | BFS | DFS |
|---|---|---|
| Adjacency list | O(n + e) | O(n + e) |
| Adjacency Matrix | $O(n^2) \sim \Theta(n^2)$ | $O(n^2) \sim \Theta(n^2)$ |